Maths Concepts & Qns

Video-6

codestorywithmik

CSwithMIK

codestorywithMIK

Motivation :-

A goal without a plan is just a wish, but a plan without action is just a daydream.

Break your long-term ambitions into small, manageable tasks and tackle them with relentless focus.
The clarity of a good plan gives you the confidence to execute when things get tough.

MIK...

Square Root Decomposition

0	1	2	3	4	5	6	7	8
2,	3,	1,	2,	5,	6,	8,	1,	3

$\{2, 3, 1, 2, 5, 6, 8, 1, 3\}, n = 9$

Query-1 : $L = 0$, $R = 8$ $\rightarrow$ for ($i = L$; $i <= R$; i++)

Query-2 : $L = 0$, $R = 8$ $\rightarrow$ " "

Q → . . .

✗ T.C = O(Q * n) TLE.

Sq. root Dec.

O(Q * n)

↓

O(Q * √n)

$\sqrt{n} = \sqrt{9} = 3$ n = 9

0	1	2	3	4	5	6	7	8
2,	3,	1,	2,	5,	6,	8,	1,	3

n = 9

i/blocksize = 5/3 = 1

blocksize = √n = 3

i = 7/3

7/3 = 2

blocks =

B0	B1	B2
2+3+1	2+5+6	8+1+3

$\{1, 3, 2, 4, 3, 2, 1, 4, 2, 2, 1, 1, 2, 3, 1, 5\}$, $n = 16$

$\sqrt{n} = 4$

$\sqrt{n}$

blocksize $= \sqrt{n} = 4$

blocks =

B0	B1	B2	B3

$\{1, 2, 3, 4, 6, 7, 2, 4\}$, $n = 8$

$\sqrt{n} = \sqrt{8} = 2\ldots$

$= \lceil 2\ldots \rceil$

blocksize = ceil(sqrt(n)) $= 3$

$= \text{ceil}(\sqrt{8})$

$= \text{ceil}(2\ldots)$

$= 3$

blocks =

B0	B1	B2
1+2+3	4+6+7	2+4

Let's Handle Queries now ...

$\{2, 3, 1, 2, 5, 6, 8, 1, 3\}$, $n = 9$

(indices: 0 1 2 3 4 5 6 7 8)

$L = 0$, $R = 8$

blocksize $= \lceil \sqrt{n} \rceil = 3$

(0, 2) (3, 5) (6, 8)

B0 B1 B2

blocks = | 2+3+1 | 2+5+6 | 8+1+3 | $\rightarrow \sqrt{n}$

6 + 13 + 12

blocks.size() $= \sqrt{n}$

$\{2, 3, 1, 2, 5, 6, 8, 1, 3\}$, $n = 9$

L R

$L = 0$, $R = 5$

blocksize = 3

(0,2) (3,5)

	B0	B1	B2
blocks =	2+3+1	2+5+6	8+1+3

G + B = 19

Handling Partial Block Queries

$\sqrt{n}$

L R

0	1	2	3	4	5	6	7	8
2,	3,	1,	2,	5,	6,	8,	1,	3

$\}, n = 9$

Case-1 : L & R is in the same block

blocksize = 3

Case-2 : L & R are in different blocks (Complete Block, Partial Block)

	B0	B1	B2
blocks =	2+3+1	2+5+6	8+1+3

```
for (i = L; i <= R; i++) → √n
    sum += nums[i];
```

case-2 → L & R are in different blocks.

B0	B1	B2
0, 1, 2	3, 4, 5	6, 7, 8
L		R
2, 3, 1,	2, 5, 6,	8, 1, 3 } , n = 9
√n		√n

blocksize = $\sqrt{9} = 3$

left partial sum → middle full blocks sum
→ Right Partial sum

	B0 (0)	B1 (1)	B2 (2)
blocks =	2+3+1	2+5+6	8+1+3

L/blsize

(•) L to end index of Block 0 → for(L to 2)

(•) Middle blocks sum = Block1

(•) B2 start index to R sum. → for(6 to R)

R/Bl.size

Block's start index :

B1 start index = 1 * 3 = 3

B2 start index = 2 * 3 = 6

B1 end index = (B2 start index) − 1 ;

B0 end index = (B1 start index) − 1 ;

= (1 * 3) − 1

= 2

B0 B1 B2 B3

L R

{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 }

$\sqrt{n}$ 1 1 $\sqrt{n}$

$n = 16$

$bl.si = \sqrt{n} = 4$

B1 to B2 → O(1).

blocks[B1] =

blocks[B2] =

```
int QuerySumRange(int L, int R)
```

```
//case-1 (L & R are in same block)

int startBlock = L/blocksize;
int endBlock = R/blocksize;

if (startBlock == endBlock) {
        for (i = L ; i<=R ; i++)        → √n
            sum += nums[i];

}
```

```
//case-2 (Both L & R are in diff. Block).

//Left (L to end of startBlock)
int endOfStartBlock = ((startBlock+1) * blocksize) -1;
for (i = L; i<= endOfStBlo ; i++)  sum += nu[i];

// middle Blocks
//(startBlock+1) to (endBlock-1)

√n ⇐ for( int b = startBlo+1; b<=endBlo-1; b++) {
        sum += blocks[b]; //O(1)

}

// Right Part (startIndex of endBlock to R)

startOfEndBlock = (endBlock * blocksize);
```

R

int startBlock;

```
for ( int i = startBlock ; i<=R ; i++)
        sum += nums[i];
```

$O(\sqrt{n})$.

Leetcode → 307.

When to use Sqrt ?

① Faster than Brute Force. $Q \cdot n \rightsquigarrow Q \cdot \sqrt{n}$

② array/nums not bejdh updated heavily.

L, R → update(+5), → Segment Tree.

③ ~~Segment tree not required~~ → complex impl.

$a \cdot \sqrt{n}$ →

Quick imple → Time saving

update(i, val)

i = 4, val = 10

i/blocks = 4/3 = 1

{1, 2, 3, 4, 10, 6, 7, 8, 9} n = 9

O(1)

	B0	B1	B2
block	6	4+5+6	24

15 − 5 = 10

10 + 10 = 20